

02 • Person 2 — Visualisation and Interface (v2)

Goal: a Streamlit app where a coach picks a pass play and watches the ball in the air, the real defender (red) and the optimal Ghost Defender (gray) racing to the catch point, with Wasted Yards in large text above the field.

What changed in v2: the data ends when the pass arrives, so the animation now runs from the throw to the arrival. There is a fourth element (the football), the blue dot is the targeted receiver, and the gray X marks the catch point. The CSV columns changed; see the v2 contract in `CLAUDE.md`.

Files you own: `field.py`, `app.py`, `make_mock_data.py`. Never import Person 1's code.

If you have already done some v1 prompts: every v2 prompt below says "create or update", so run it whatever state your files are in. `field.py` (P2-1) does not change.

Order: P2-0 → P2-5. Commit after each one passes its check.

P2-0 • Mock data in the v2 format

text

Task: create or rewrite make_mock_data.py so it writes data/mock/ghost_frames.c

Implement the ghost formula from CLAUDE.md "Ghost path" and the gaps from "Wast

Play 1 (gameId 1, playId 1): the "Golden play" from CLAUDE.md v2.

- frameIds 1–21, throw on frameId 1, arrival on frameId 21, arrival_event "pass
- Simulate the receiver, the real defender and the football exactly as describe
- Names "Mock Receiver" (nflId 101) and "Mock Defender" (nflId 201, position CB

Play 2 (gameId 1, playId 2): a perfect angle.

- Same start and same receiver and football motion as play 1.
- The real defender follows the ghost's path exactly.
- pass_result "I". description "MOCK – optimal angle, pass broken up".

Play 3 (gameId 2, playId 1): out of range.

- 2.5 s of air time: frameIds 1–26, 25 steps.
- QB at (40, 26). Receiver starts at (50, 10), runs dir 90 at 9.0 yd/s, so the
- Defender starts at (60, 40), top speed 7.0 yd/s, runs straight at the catch p
- pass_result "C". description "MOCK – deep ball, defender out of range".

Compute every summary column from the simulated positions. Do not hard-code res

Run it and print the summary table. Required results:

- Play 1: catch (64, 12), ghost_reach_time 1.325, ghost_gap 0.00, real_gap 5.40
 - Play 2: wasted_yards 0.00.
 - Play 3: ghost_reached False, ghost_reach_time empty, ghost_gap 15.00, real_ga
- If any value is off, fix the simulation, not the expected values.

Check: the three plays match the required results.

P2-1 • The football field

No change from v1. If `field.py` is done and `field_preview.html` looks right, skip this.
Otherwise run the v1 P2-1 prompt as written (trace 0 is the hash-mark trace named "field").

P2-2 • App skeleton: data, sidebar, metric

text

Task: create or update app.py for the v2 contract: data loading, sidebar and th

1. load_data() with @st.cache_data:
 - if data/processed/ghost_frames.csv and data/processed/play_summary.csv bot
 - otherwise load data/mock/ and return source "mock"
 - returns (frames, summary, source)
2. Page: st.set_page_config(layout="wide", page_title="Ghost Defender"). Title
3. If source is "mock", show st.warning at the top: "MOCK DATA – run data_proce
4. Sidebar:
 - selectbox gameId (sorted)
 - selectbox playId, filtered to that game. Label each option "Play 1 · 5.4 w
 - below: the description, pass_result and coverage, in small text
5. Header above the field:
 - Wasted Yards as the biggest element on the page: a custom st.markdown bloc
 - Colour: under 0.5 → green with "≈ optimal angle". 0.5–2 → amber. Over 2 →
 - If ghost_reached is False: show the number in gray, with the line "Out of
 - Next to it, st.metric tiles: Gap at arrival (real_gap, yd), Ghost gap (gho
 - Under them: "Defender: <defender_name> (<defender_position>) · Receiver: <
6. Below the header, if no animation exists yet: st.plotly_chart(draw_field())
7. Remove any references to v1 columns (intercept_x, t_intercept, reached_carri

Run `streamlit run app.py`, read the terminal for errors, and tell me exactly w

Check: play 1 shows red 5.4; play 2 shows green 0.0 "≈ optimal angle"; play 3 shows gray 0.0 with the out-of-range line.

P2-3 • The animation

Put **Plan first: list the functions you will write and their signatures, then wait for my OK.** at the front.

text

Task: create or rewrite `build_play_figure(play_frames, summary_row, frame_ms=10`

Start from `draw_field()` (trace 0 is the field). Add traces in exactly this order

1. Ghost planned path: static dashed gray line from `(ghost_start_x, ghost_start_y)` to `(catch_x, catch_y)`, size 14, hover "
2. Catch point: static white "x" marker at `(catch_x, catch_y)`, size 14, hover "
3. Real defender trail: red solid line from the first frame up to the current frame
4. Receiver trail: thin blue line, same idea.
5. Ghost Defender dot: gray #9e9e9e, opacity 0.5, size 16, white outline.
6. Real Defender dot: red #d62728, size 16.
7. Receiver dot: blue #1f77b4, size 16.
8. Football dot: brown #8b4513, size 10, white outline.

Moving dots come last so they draw on top.

Animation:

- One `go.Frame` per `frameId`, in sorted order, `name = str(frameId)`, containing data for that frame
- The figure's initial data for traces 3–8 equals the first frame.
- Play and Pause buttons (`layout.updatemenus`): frame duration `frame_ms`, transition `frame_ms`
- A slider under the field, one step per frame, labelled with the `t` column, e.g. "0.0 s"
- Legend at the top, horizontal, showing only: "Receiver", "Real defender", "Ghost defender", "Football"
- Hover on dots: `displayName` and `t`.
- Keep the axis ranges fixed during animation. Set them explicitly; never autorange

Run `streamlit` and read the terminal for errors. Then tell me exactly what I should see

Check: on mock play 1, the red dot first runs the wrong way (towards the line of scrimmage) for about 0.8 s, then turns towards the X. The gray dot runs straight to the X and sits there from about $t = 1.3$ s. The ball lands on the blue dot at $t = 2.0$ s with the red dot well short. On play 2, red and gray move together.

→ **Checkpoint B:** when Person 1's CSVs arrive, go to `03_INTEGRATION_AND_DEMO.md`.

P2-4 · Coach-facing polish

text

Task: polish app.py. Do not edit any other file.

1. Sidebar radio "Playback speed": 0.25x, 0.5x, 1x → frame_ms 400, 200, 100. De
2. Sidebar checkbox "Show final frame (for screenshot)". When ticked, render a
3. Plotly config for st.plotly_chart: displaylogo False, and toImageButtonOptio
4. An expander "How to read this" under the chart, with three short lines:
 - Blue is the targeted receiver and brown is the ball. Red is the real defen
 - The white X is where the ball arrived.
 - Wasted Yards = how much farther from the catch point the real defender was
5. Make sure switching plays resets the animation to the first frame, with no l

Run it and check the terminal for errors.

Check: tick the final-frame box and click the camera icon on the chart. You should get a sharp PNG with the annotation visible.

P2-5 • Leaderboard tab (stretch)

text

Task: add tabs to app.py: "Play viewer" (everything built so far) and "Worst an

In "Worst angles":

1. A table of the 25 plays with the largest wasted_yards, among plays where gho
2. A defender leaderboard: group by defender_name, keep defenders with at least
3. Above the tables, a selectbox "Open play" listing the top 25. Choosing one s

It must still work on mock data, where the tables will be short.

Check: pick a play in "Open play", switch tabs, and the viewer shows that play.